

UNIVERSIDAD CENTRAL DE VENEZUELA
FACULTAD DE CIENCIAS
ESCUELA DE COMPUTACION
ADMINISTRACION DE BASES DE DATOS

PROYECTO #1 — STREAMUCV
INFORME TECNICO DE LA SOLUCION

Grupo Docente:
Br. Joiner Rojas
Br. Maximiliano Barboza

Integrantes:
Ronald Herrera
Brhandon Palomo
Sebastián Russian

Caracas, Junio de 2026

Índice

1. Descripcion General de la Solucion del Proyecto	2
1.1. Arquitectura de la Solucion	2
1.2. Beneficios del Diseno	2
2. Explicacion del Uso del Diccionario de Datos	3
2.1. R1: Listado de Tablas e Indices del Esquema Streaming	3
2.2. R2: Conteo Total de Tablas y Cantidad de Indices por Tabla	3
2.3. R3: Restricciones Existentes en el Esquema	3
2.4. R4: Columnas que Conforman cada Indice	4
2.5. R5: Triggers Activos en el Esquema	4
2.6. R6: Tamano Fisico Ocupado por cada Tabla	4
2.7. R7: Estimacion del Tamano del Registro en Bytes	5
2.8. R8: Tamano de cada Columna segun su Tipo de Dato	5
2.9. R9: Calculo del Factor de Bloqueo	5
2.10. R10: Estimacion del Costo y Tiempo para Consultas de Igualdad	6
2.10.1. 1. Deteccion de Indices Utilizables	6
2.10.2. 2. Caso A: Acceso Secuencial (Full Table Scan)	6
2.10.3. 3. Caso B: Acceso por Indice (Index Seek)	6
3. Instrucciones de Ejecucion Detalladas	8
3.1. Paso 1: Instalacion de Prerrequisitos	8
3.2. Paso 2: Instalacion de las Dependencias de Python	8
3.3. Paso 3: Creacion y Poblado de la Base de Datos	8
3.4. Paso 4: Configuracion de Parametros de Conexion	9
3.4.1. Autenticacion de Windows (Windows Auth)	9
3.5. Paso 5: Lanzamiento de la Aplicacion	9
3.6. Paso 6: Uso y Navegacion de la Aplicacion	9

1. Descripcion General de la Solucion del Proyecto

El proyecto **StreamUCV** es una solucion modular e interactiva orientada a resolver un problema critico del departamento de Analisis de Datos de la empresa: la falta de visibilidad estructurada sobre su base de datos alojada en **Microsoft SQL Server 2022**. Para ello, se ha disenado y desarrollado una aplicacion en **Python** utilizando el framework web **Streamlit**. Esta aplicacion interroga en tiempo real el **Diccionario de Datos (D/D)** de la instancia y presenta un reporte estructurado y visual de la metadata del sistema.

1.1. Arquitectura de la Solucion

La solucion se compone de varias capas bien diferenciadas, respetando el principio de separacion de responsabilidades:

- **Capa de Punto de Entrada (main.py)**: Es el archivo que sirve como iniciador simplificado de la aplicacion. Configura la pagina de Streamlit y cede el control a la interfaz grafica.
- **Capa de Configuracion (app/config.py)**: Centraliza todas las variables globales del sistema requeridas por el enunciado, incluyendo la cadena de conexion por defecto (**SERVER**, **DATABASE**, **USERNAME**, **PASSWORD**, **DRIVER**) y los parametros de computo fisico (tamano de la pagina de datos a 8192 bytes y la velocidad de transferencia del disco a 17 MB/s).
- **Capa de Conexion (app/connection.py)**: Utiliza la biblioteca **pyodbc** para gestionar de manera segura la apertura y el cierre de conexiones a SQL Server. Soporta de forma dinamica tanto la autentificacion nativa de SQL Server (usuario/contrasena) como la autentificacion integrada de Windows (**Trusted.Connection=yes**), e incluye la gestion del cifrado SSL (**Encrypt=yes**) para drivers modernos.
- **Capa de Consulta al Diccionario de Datos (app/queries/)**: Contiene 10 modulos independientes en Python (del **r01_tablas_indices.py** al **r10_costo_consulta.py**). Cada script implementa de forma aislada las consultas SQL especializadas sobre las tablas del sistema de SQL Server y los calculos asociados. Los resultados se procesan como objetos **pandas.DataFrame**.
- **Capa de Interfaz de Usuario (app/ui/streamlit_app.py)**: Implementa el panel de control web. Dispone de una barra lateral interactiva para modificar en caliente los parametros de conexion, garantizando una excelente portabilidad. Asimismo, divide los 10 requerimientos del sistema en pestanas y controles interactivos que facilitan la navegacion de los analistas de datos.

1.2. Beneficios del Diseno

1. **Portabilidad Absoluta**: La base de datos y la cadena de conexion no estan codificadas de forma rigida en multiples puntos del programa. Se definen en **config.py** y se pueden sobrescribir dinamicamente desde la barra lateral de la interfaz. Esto asegura que la aplicacion pueda ejecutarse en cualquier equipo sin modificar el codigo fuente.
2. **Eficiencia en la Consulta**: La aplicacion no lee datos de usuario directamente para calcular estadisticas fisicas; en su lugar, accede a las vistas del sistema catalogadas de SQL Server, lo que minimiza la carga de trabajo sobre el motor relacional.
3. **Interactividad Avanzada**: En lugar de requerir que los analistas ejecuten sentencias SQL complejas manualmente, la aplicacion provee botones, tablas dinamicas interactivos y selectores que estiman los costos fisicos de forma automatizada.

2. Explicacion del Uso del Diccionario de Datos

El Diccionario de Datos (D/D) o catalogo de base de datos es una coleccion de tablas internas y vistas de solo lectura que almacenan los metadatos del motor SQL Server (es decir, datos sobre la estructura fisica y logica de la base de datos).

En lugar de consultar tablas de usuario, esta solucion interroga las vistas de catalogo internas en el esquema `sys`. A continuacion se explica detalladamente la logica, las tablas de sistema utilizadas y los supuestos de calculo matematico implementados para cada uno de los 10 requerimientos funcionales del proyecto.

2.1. R1: Listado de Tablas e Indices del Esquema Streaming

El objetivo es obtener una lista detallada de todas las tablas e indices definidos en el esquema `streaming`.

- **Tablas del sistema utilizadas:**
 - `sys.tables (t)`: Contiene una fila por cada objeto de tabla en la base de datos.
 - `sys.schemas (s)`: Almacena informacion sobre los esquemas. Permite filtrar para obtener unicamente las tablas del esquema `streaming`.
 - `sys.indexes (i)`: Contiene una fila por cada indice de cada tabla.
- **Logica SQL:** Para obtener las tablas, se une `sys.tables` con `sys.schemas` bajo la condicion `s.name = 'streaming'`. Para los indices, se anade una junta con `sys.indexes` filtrando `i.name IS NOT NULL` (evitando asi la fila por defecto con `index_id = 0` que representa la tabla organizada en monton o Heap).

2.2. R2: Conteo Total de Tablas y Cantidad de Indices por Tabla

Este requerimiento indica el volumen total de la base de datos en terminos de estructuras logicas.

- **Tablas del sistema utilizadas:** `sys.tables`, `sys.schemas` y `sys.indexes`.
- **Logica SQL:** Se realiza un conteo escalar de tablas en el esquema `streaming`. A su vez, para el desglose por tabla, se ejecuta un `LEFT JOIN` entre `sys.tables` y `sys.indexes` agrupando por `t.name`, contando los identificadores de indice validos.

2.3. R3: Restricciones Existentes en el Esquema

Permite auditar las reglas de integridad fisica declaradas en el modelo relacional.

- **Tablas del sistema utilizadas:**
 - `sys.objects (c)`: Almacena metadatos de todos los objetos de la base de datos.
 - `sys.tables (t)` y `sys.schemas (s)`.
- **Clasificacion de Restricciones:** Se filtran los objetos en `sys.objects` segun su propiedad `c.type`: Se une con la tabla padre mediante la relacion `c.parent_object_id = t.object_id`.

Codigo de Tipo	Descripcion de Restriccion
PK	Clave Primaria (Primary Key)
F	Clave Foranea (Foreign Key)
C	Restriccion de Verificacion (Check Constraint)
UQ	Restriccion de Unicidad (Unique)
D	Valor por Defecto (Default Constraint)

Cuadro 1: Mapeo de restricciones en `sys.objects`.

2.4. R4: Columnas que Conforman cada Indice

Muestra de forma compacta que atributos del modelo logico componen la clave de busqueda de cada indice configurado.

- **Tablas del sistema utilizadas:** `sys.indexes`, `sys.index_columns` (`ic`), `sys.columns` (`c`), `sys.tables` y `sys.schemas`.
- **Logica SQL de Concatenacion:** Dado que un indice puede ser compuesto (multiples columnas), se emplea la funcion `STRING_AGG` ordenando los elementos por `ic.key_ordinal`. Ademas, se verifica la propiedad `ic.is_descending_key` para agregar el sufijo `ASC` o `DESC` a la columna, proporcionando informacion precisa de ordenamiento fisico.

2.5. R5: Triggers Activos en el Esquema

Lista el comportamiento activo automatizado (disparadores) sobre las tablas del sistema.

- **Tablas del sistema utilizadas:** `sys.triggers` (`tr`), `sys.trigger_events` (`te`) y `sys.tables` (`t`).
- **Logica SQL:** Los triggers se clasifican segun su tipo de activacion consultando `tr.is_instead_of_trigger` (retorna `INSTEAD OF` si es verdadero, o `AFTER` por defecto). El estado de habilitacion se extrae de `tr.is_disabled`. Los eventos asociados (`INSERT`, `UPDATE`, `DELETE`) se obtienen agrupando por `sys.trigger_events`.

2.6. R6: Tamano Fisico Ocupado por cada Tabla

Para determinar el espacio en almacenamiento persistente de cada objeto, se debe interrogar las estadísticas físicas de paginación del motor de base de datos.

- **Tablas del sistema utilizadas:**
 - `sys.tables` y `sys.schemas`.
 - `sys.dm_db_partition_stats` (`ps`): Vista de administracion dinamica que expone el conteo de paginas de datos y filas por particion de objeto.
- **Logica de Computo Espacial:** SQL Server organiza los datos fisicamente en paginas de **8 KB**. Para calcular el tamano real:
 - **Filas totales:** Suma de `ps.row_count` donde `ps.index_id` es 0 (Heap) o 1 (Clustered Index).
 - **Datos (KB):** Paginas usadas por el monton o indice agrupado multiplicado por 8:

$$\text{Datos (KB)} = \sum_{\text{index_id} \in \{0,1\}} \text{ps.used_page_count} \times 8$$

- **Indices (KB):** Paginas usadas por los indices no agrupados adicionales (`index_id > 1`) multiplicado por 8:

$$\text{Indices (KB)} = \sum_{\text{index.id} > 1} \text{ps.used_page_count} \times 8$$

- **Total Usado (KB):** Suma total de las paginas utilizadas de la tabla por 8.

2.7. R7: Estimacion del Tamano del Registro en Bytes

Calcula de forma teorica cuanto ocupa un registro (tupla) completo de cada tabla en memoria o disco.

- **Formula de Estimacion:** De acuerdo con el enunciado, para efectos de uniformidad, el tamano estimado del registro de una tabla T se calcula sumando la longitud maxima fisica declarada de todas sus columnas constituyentes:

$$S_{reg}(T) = \sum_{c \in T} c.\text{max_length} \quad (1)$$

- **Tablas del sistema utilizadas:** `sys.columns` (c), `sys.tables` (t) y `sys.schemas` (s).
- **Justificacion Fisica:** El atributo `max_length` en la vista `sys.columns` representa el tamano de bytes fisico que el tipo de datos reserva en el registro. Por ejemplo, un tipo `VARCHAR(50)` tiene una longitud maxima de 50 bytes, mientras que un tipo `Unicode NVARCHAR(50)` ocupa fisicamente 100 bytes (2 bytes por caracter). Esto asegura estimaciones realistas basadas en el almacenamiento en paginas de disco.

2.8. R8: Tamano de cada Columna segun su Tipo de Dato

Permite desglosar de forma pormenorizada las propiedades fisicas de las columnas de cada tabla.

- **Tablas del sistema utilizadas:** `sys.columns`, `sys.types` (tp), `sys.tables` y `sys.schemas`.
- **Logica SQL para la Definicion de Tipos:** Se utiliza una expresion condicional CASE para reconstruir la sintaxis SQL de tipos:
 - Para `varchar` y `char`: Se muestra la longitud en bytes directa de `max_length`.
 - Para `nvarchar` y `nchar`: Se muestra la longitud logica de caracteres dividiendo entre 2 (`max_length/2`).
 - Para `decimal` y `numeric`: Se concatenan la precision y la escala declaradas: `tp.name(precision, scale)`.

2.9. R9: Calculo del Factor de Bloqueo

El **Factor de Bloqueo (Blocking Factor, FB)** representa el numero maximo de registros completos que pueden almacenarse dentro de un bloque fisico de datos (pagina de datos).

- **Supuesto de Computo:** Los registros son de longitud fija y no se extienden a lo largo de varias paginas.
- **Constante Fisica:** En SQL Server, una pagina de datos tiene un tamano fijo de **8192 bytes** (8 KB).

- **Formula de Factor de Bloqueo para Tablas:**

$$FB_{\text{tabla}} = \left\lfloor \frac{8192}{\text{Tamano Estimado del Registro en Bytes}} \right\rfloor = \left\lfloor \frac{8192}{S_{\text{reg}}} \right\rfloor \quad (2)$$

- **Formula de Factor de Bloqueo para Indices:** Para los indices, el factor de bloqueo se calcula determinando primero la longitud de la clave del indice. Esta longitud corresponde a la suma de los tamanos fisicos de las columnas que conforman la clave indexada:

$$S_{\text{clave.indice}} = \sum_{c \in \text{Claves}} c.\text{max_length} \quad (3)$$

El factor de bloqueo de indice (FB_{indice}) se define como:

$$FB_{\text{indice}} = \left\lfloor \frac{8192}{S_{\text{clave.indice}}} \right\rfloor \quad (4)$$

2.10. R10: Estimacion del Costo y Tiempo para Consultas de Igualdad

Este requerimiento evalua el impacto del uso de indices en consultas de busqueda exacta (`WHERE columna = valor`).

2.10.1. 1. Deteccion de Indices Utilizables

Para que un indice sea utilizable en una busqueda de igualdad sobre una columna dada, dicha columna debe ser el **primer atributo de ordenacion en la estructura del indice**. Logicamente, esto se detecta en la tabla `sys.index_columns` verificando que `key_ordinal = 1`.

2.10.2. 2. Caso A: Acceso Secuencial (Full Table Scan)

Se produce si no existe ningun indice aplicable a la consulta.

- **Costo en Accesos a Disco:** Se lee secuencialmente la tabla completa desde el disco. El costo es igual al total de paginas usadas por los datos de la tabla (P):

$$C_{\text{scan}} = P \quad (5)$$

- **Costo en Tiempo:** Utilizando la velocidad de transferencia del disco de **17 MB/s** ($17 \times 1024 \times 1024$ bytes por segundo), calculamos el tiempo requerido para transferir P paginas a memoria:

$$T_{\text{scan}} = \frac{C_{\text{scan}} \times 8192 \text{ bytes}}{17 \times 1024 \times 1024 \text{ bytes/seg}} \approx C_{\text{scan}} \times 0,0004595 \text{ segundos} \quad (6)$$

2.10.3. 3. Caso B: Acceso por Indice (Index Seek)

Si se encuentra un indice donde la columna consultada sea el atributo de clave principal.

- **Altura Estimada del Arbol B-Tree (h):** La altura del arbol de indices regula la cantidad de lecturas de nodos directorio antes de alcanzar el nodo hoja. Academicamente, asumiendo un factor de ramificacion (**fan-out**) promedio de $F = 100$ entradas por nodo de indice, la altura se estima como:

$$h = \text{máx}(1, \lceil \log_{100}(P) \rceil) \quad (7)$$

- **Costo en Accesos a Disco para Clustered Index (Indice Agrupado):** En un indice agrupado, las paginas hoja del indice son las mismas paginas de datos de la tabla. Al llegar al nivel hoja, se encuentra el registro directamente. El costo es recorrer la altura del B-Tree mas leer la pagina de datos final:

$$C_{\text{seek, clustered}} = h + 1 \quad (8)$$

- **Costo en Accesos a Disco para Non-Clustered Index (Indice No Agrupado):** En un indice no agrupado, las paginas hoja contienen punteros fisicos (o la clave de ordenacion agrupada) que apuntan al registro real en la tabla de datos. Por lo tanto, el sistema debe recorrer el arbol de indice y realizar un acceso adicional a disco (Lookup) para recuperar la tupla de datos real:

$$C_{\text{seek, non-clustered}} = h + 1 \text{ (Index Seek)} + 1 \text{ (Data Page Lookup)} = h + 2 \quad (9)$$

- **Acotacion de Costo:** En tablas muy pequenas (diminutas), el costo estimado de un seek podria ser superior al numero total de paginas fisicas de la tabla (P). En la practica, el optimizador de consultas preferira un escaneo secuencial en estos casos. Por ende, la aplicacion acota el costo indexado:

$$C_{\text{seek}} = \min(C_{\text{seek, calculado}}, C_{\text{scan}}) \quad (10)$$

- **Costo en Tiempo:** El calculo del tiempo para la busqueda indexada se deduce multiplicando los accesos a disco por el tiempo de lectura de pagina fisica:

$$T_{\text{seek}} = \frac{C_{\text{seek}} \times 8192 \text{ bytes}}{17 \times 1024 \times 1024 \text{ bytes/seg}} \approx C_{\text{seek}} \times 0,0004595 \text{ segundos} \quad (11)$$

3. Instrucciones de Ejecucion Detalladas

Esta seccion detalla los pasos requeridos para preparar el entorno de base de datos, configurar la conexion logica de la aplicacion y ejecutar la interfaz de usuario en cualquier equipo local.

3.1. Paso 1: Instalacion de Prerrequisitos

Antes de iniciar, es indispensable que el sistema cuente con los siguientes componentes:

1. **Python 3.11 o superior:** Verifique la version instalada ejecutando en la terminal:

```
1 python --version
2
```

2. **Microsoft SQL Server:** Debe estar activo y accesible en la red local o mediante un contenedor Docker.
3. **Driver ODBC de SQL Server:** El sistema operativo debe tener instalado el driver oficial de Microsoft. Las versiones compatibles son:
 - ODBC Driver 17 for SQL Server (Recomendado y configurado por defecto)
 - ODBC Driver 18 for SQL Server

3.2. Paso 2: Instalacion de las Dependencias de Python

Abra su terminal o consola de comandos en la raiz del proyecto (directorio **Fase-1**) e instale las bibliotecas listadas en `requirements.txt` utilizando el gestor de paquetes `pip`:

```
1 pip install -r requirements.txt
```

Este comando instalara los modulos esenciales:

- **Streamlit:** Para renderizar la interfaz grafica web interactiva.
- **pyodbc:** Para realizar la conexion nativa a la base de datos SQL Server.
- **pandas:** Para la manipulacion, limpieza y visualizacion en tablas de los DataFrames resultantes.

3.3. Paso 3: Creacion y Poblado de la Base de Datos

Para facilitar la creacion de la base de datos de pruebas **StreamUCV** y su esquema de trabajo **streaming**, se incluye un script integrador automatico llamado `setup_db.py` que ejecuta ordenadamente los scripts SQL provistos en la carpeta `scripts/`:

1. `create_repositorio_sqlserver.sql`: Crea la base de datos vacia **StreamUCV** y establece el esquema **streaming**.
2. `create_tables_sqlserver.sql`: Crea todas las tablas fisicas del esquema, sus respectivas claves primarias, claves foraneas, restricciones de verificacion (**CHECK**) e indices asociados.
3. `insert_tables_sqlserver.sql`: Realiza la carga masiva de registros de prueba (series, peliculas, artistas, personajes, horarios, ventas, etc.) requeridos para el correcto funcionamiento del reporte.

Para ejecutar el configurador automatizado, corra el siguiente comando:

```
1 python setup_db.py
```

Nota: Si la conexion a la instancia maestra de SQL Server falla, asegurese de que el servidor por defecto en `app/config.py` coincida con su instancia local (por ejemplo, `localhost\SQLEXPRESS`).

3.4. Paso 4: Configuración de Parametros de Conexión

Para garantizar la portabilidad, el archivo `app/config.py` define los valores globales por defecto. Puede editar este archivo para adaptarlo permanentemente a su infraestructura:

```
1 # app/config.py
2 SERVER = "localhost\\SQLEXPRESS"    # Su servidor o instancia
3 DATABASE = "StreamUCV"              # Base de datos
4 USERNAME = ""                       # Deje vacio si usa Windows Auth
5 PASSWORD = ""                       # Contraseña del usuario SQL
6 DRIVER = "ODBC Driver 17 for SQL Server"
```

3.4.1. Autenticación de Windows (Windows Auth)

Si se deja el usuario y la contraseña como cadenas vacías (), la capa de conexión configurará automáticamente la propiedad `Trusted_Connection=yes`. Esto significa que heredará las credenciales del usuario actual del sistema operativo Windows, eliminando la necesidad de exponer contraseñas en texto claro.

3.5. Paso 5: Lanzamiento de la Aplicación

Para iniciar la interfaz interactiva con el servidor de desarrollo Streamlit, ejecute el siguiente comando en la raíz del proyecto:

```
1 streamlit run main.py
```

De manera automática, se iniciará el servidor local y se abrirá una ventana en su navegador web predeterminado cargando la interfaz gráfica (típicamente en la dirección <http://localhost:8501>).

3.6. Paso 6: Uso y Navegación de la Aplicación

1. **Barra Lateral (Sidebar):** En el panel izquierdo de la pantalla, podrá visualizar los parámetros de conexión actuales extraídos de `config.py`. Podrá editar cualquier valor (Servidor, Base de datos, Usuario, Contraseña, Driver) directamente desde la interfaz y hacer clic en el botón **"Probar Conexión"**. El sistema validará las credenciales y mostrará un mensaje de éxito o el error detallado devuelto por SQL Server.
2. **Pestanas de Requerimientos:** La pantalla principal se divide en secciones dedicadas:
 - **Estructura Lógica:** Pestanas dedicadas a listar Tablas e Índices (R1), Conteos Lógicos (R2), Restricciones de Integridad (R3) y Detalle de Índices por Columna (R4).
 - **Eventos y Disparadores:** Muestra los triggers asociados a tablas y sus tipos de activación (R5).
 - **Almacenamiento Físico:** Expone el Tamaño Ocupado por Tabla en bytes y Megabytes (R6), estimación del registro de cada tabla (R7) y detalles de tamaño por columna según el tipo de dato físico de SQL Server (R8).
 - **Factor de Bloqueo:** Visualiza los registros por bloque/página estimados de tablas e índices (R9).
 - **Costo de Consulta (R10):** Ofrece dos menús desplegables para seleccionar cualquier tabla y columna. Al presionar el botón de cálculo, la aplicación evalúa dinámicamente si existe un índice aplicable en el Diccionario de Datos, y compara el costo de accesos a disco y el tiempo de respuesta estimado entre un escaneo secuencial (*Full Table Scan*) y un acceso directo por árbol (*Index Seek*).